



Client Selection in Federated Learning

CS-503 / AI211 · Machine Learning Project · IIT Ropar

MENTOR: SHRADHA SHARMA

Venkata Praneeth J (2023CSB1296)

Potla Prathvik (2023MCB1307)

Problem Statement

The Federated Learning Setting

FL enables collaborative model training without sharing raw data. Clients train locally and share only model updates, preserving privacy at scale across 100 clients over 100 communication rounds.



Core Question: How do we select the **optimal subset of clients** each round to improve accuracy, reduce cost, and handle concept drift?

Non-IID Data

Each client holds a different local data distribution, making aggregation challenging.

Unbalanced Data

Clients vary significantly in dataset size, skewing gradient contributions.

Low-Quality Clients

Noisy or irrelevant updates degrade global model performance.

Communication Cost

Full participation is prohibitively expensive; partial selection is necessary.

Motivation



Random Selection Falls Short

Random client selection may miss critical data patterns or select redundant clients, wasting communication rounds and reducing model quality.



Concept Drift is Real

Data distributions shift over time .user behaviour, environmental changes, and sensor degradation all cause drift that static FL systems cannot handle.



Intelligent Selection Needed

We need strategies that are efficient, robust to noise, and adaptive to dynamic environments , going beyond naive participation schemes.

Methodology – I

Task 1

Set up:

- Uses MNIST Dataset with 100 clients and 100 federated rounds.
- The training data is split in a non-IID way using a Dirichlet distribution.

Client Selection Pipeline:

- At each round, the server first samples a candidate pool of clients.
- Pool size is chosen as $\max(5K, 40)$, capped by 100 clients.
- From this pool, the server selects K clients using one of three strategies: Shapley, DivFL, Random

How the Selection Methods work:

- **DivFL**: Selects clients that maximize diversity in model updates for better global representation.
- **Shapley**: Selects clients based on their contribution to improving model accuracy.
- **Random**: Selects clients uniformly at random like baseline method.

Federated learning loop

- Selected clients train locally from the current global model.
- The server aggregates their models using **FedAvg**.
- After aggregation, the global model is evaluated and accuracy is tracked across rounds.

Task 2

Task 2 introduces concept drift over time.

Label Drift:

- Labels are not changed permanently, but the class mapping/distribution seen by a client changes over time.
- Two modes are tested:
 - Sudden drift**: Distribution changes instantly at a specific round
 - Gradual drift**: Smooth transition between old and new distributions

How drift is distributed across clients :

- 100 clients are split into groups across drift periods 4, 5, 6, 7, and 8
- Each period group has 20 clients, so drift is staggered rather than happening to all clients at once.

Training Process with Drift :

1. Update client data distribution (if drift scheduled)
2. Sample candidate pool
3. Select K clients (DivFL / Shapley)
4. Local training on updated data
5. Aggregate using FedAvg
6. Evaluate on test data

Methodology – II

Task 3a(Driftless)

Driftless Client Clustering with Single Global Model

How it Works:

- Server maintains one global model
- Selected clients from clusters based on the size of the cluster
- Server aggregates using FedAvg, then refines using FedAdam which ensures stable and efficient global learning

How Server Cluster Clients : (Warm-up + Clustering)

- Warm-up rounds collect client update vectors from all the clients
- Uses k-means clustering on updates, periodically.
- Groups clients with similar training behavior

Client Selection & Aggregation:

- Clients selected proportionally from each cluster
- Random sampling within clusters ensures diversity
- Aggregation:
 - FedAvg → compute global direction
 - FedAdam → adaptive update (momentum + scaling)

Task 3b(Drift)

Client Scoring and Multi-Model Update Framework

What the system tries to do:

- The server does not see client data directly.
- It only receives model updates and summary statistics from clients.

How the client score is calculated :

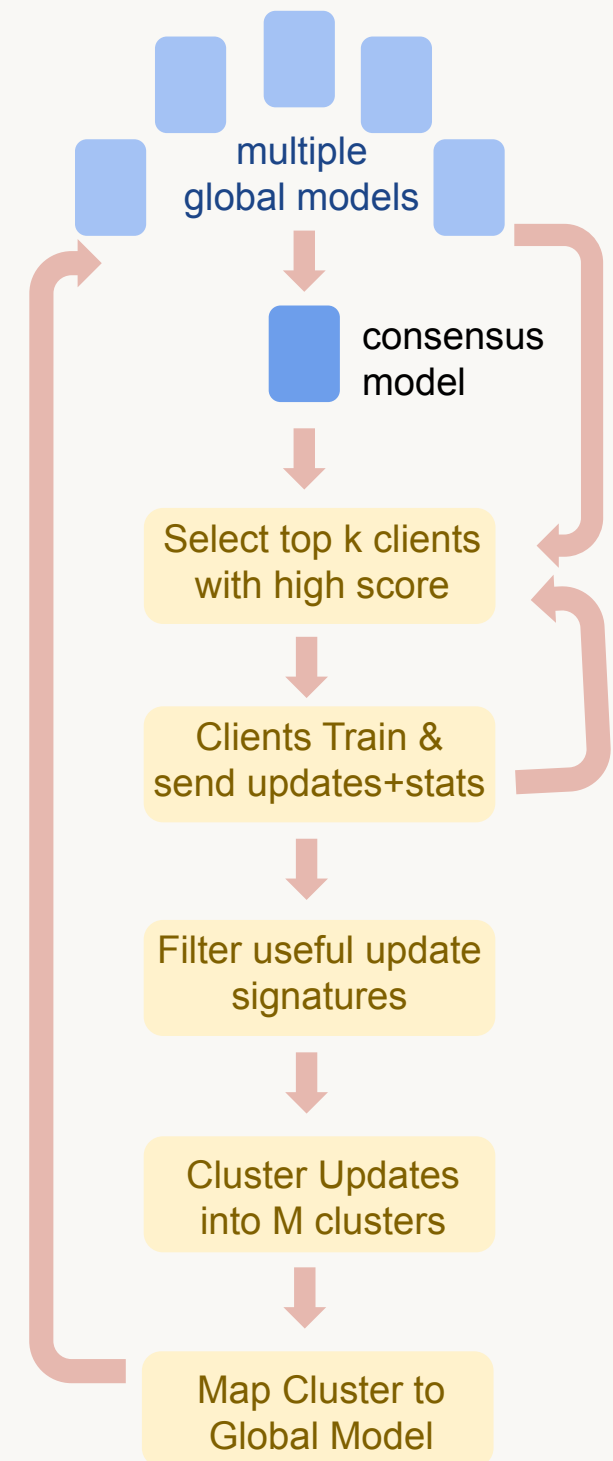
- Each client gets a score based on: Accuracy gain, Diversity score, Rare-label score, Drift score, Communication penalty, Recent-selection penalty.
- We also receive the statistics from the client along with updates

How client updates are grouped and mapped

- The Update signatures are clustered using k-means
- Each cluster represents clients with similar update behavior
- The clusters are then mapped to M global server models.
- Each mapped cluster updates using FedAvg + FedAdam.

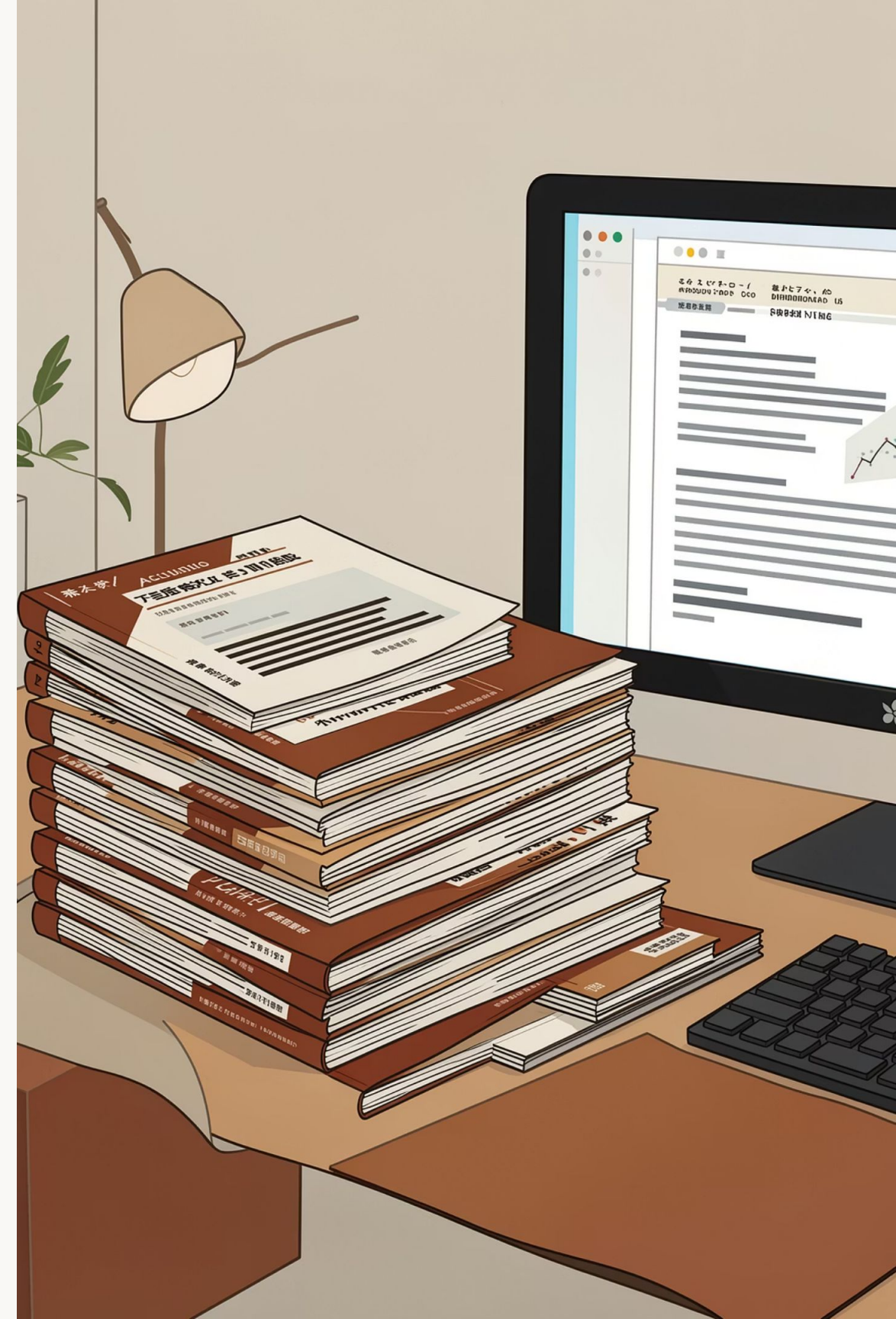
Overall workflow:

- Server builds a consensus model from all global models.
- Clients are scored and the top K are selected.
- Selected clients train locally and send updates.
- Useful Updates are filtered and Updates are clustered using k-means.
- Each cluster updates one of the M global models.

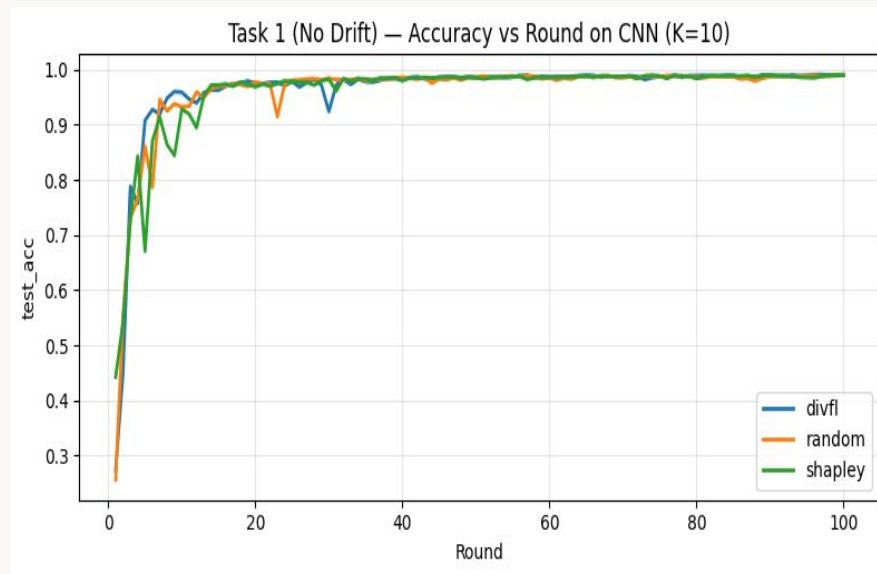


Related Work

- 1 — FedAvg (McMahan et al., 2017)
We use FedAvg as the core aggregation framework and build upon it by incorporating advanced client selection mechanisms to improve performance under non-IID settings.
- 2 — DivFL (ICLR 2022)
Applies submodular optimisation for client selection, prioritising diversity in gradient updates to reduce redundancy, improve representation, and promote fairer performance across heterogeneous clients.
- 3 — S-FedAvg (AAAI 2021)
Applies Shapley values from cooperative game theory to measure each client's contribution to model improvement, enabling top-K selection of high-impact clients.
- 4 — Concept Drift in FL
To address non-stationary data in FL, we simulate distributed concept drift in MNIST and evaluate our methods under sudden and gradual label drift, extending this with drift-aware selection in Task 3.

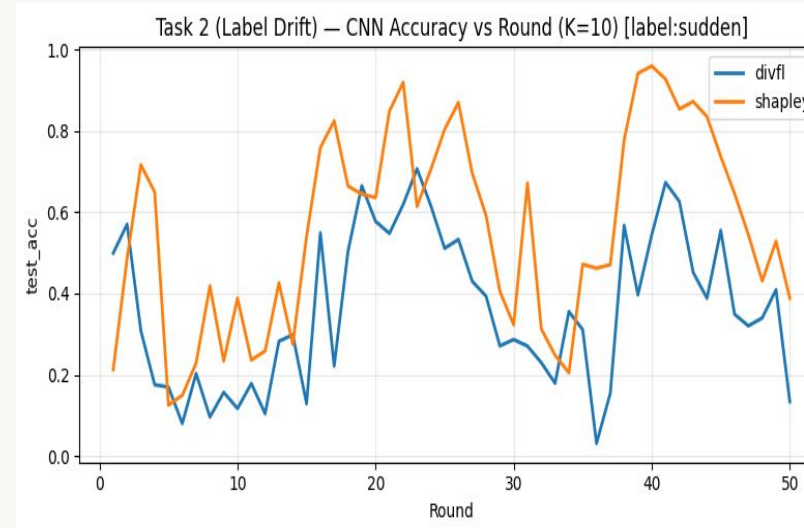


Results



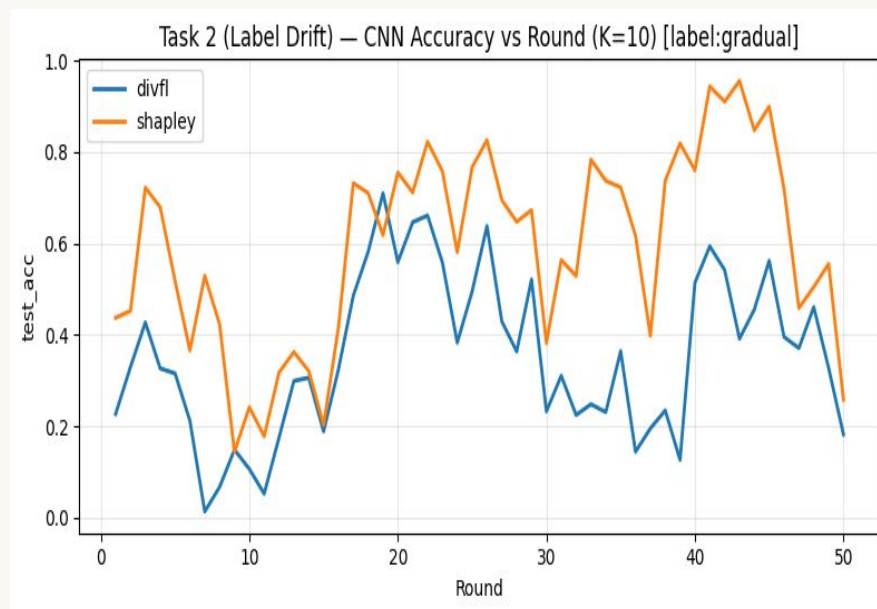
Task 1

- All methods reach ~99% (no drift)
- DivFL converges slightly faster
- Shapley shows early instability
- Differences minimal (stationary data)



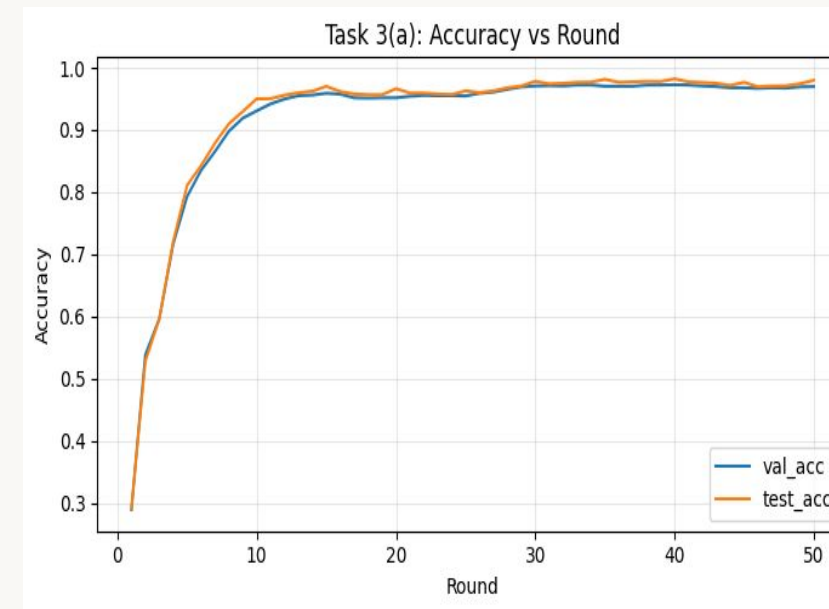
Task 2(Sudden)

- Sharp accuracy drop after drift
- Shapley recovers faster
- DivFL struggles with abrupt change
- Instability due to sudden distribution shift



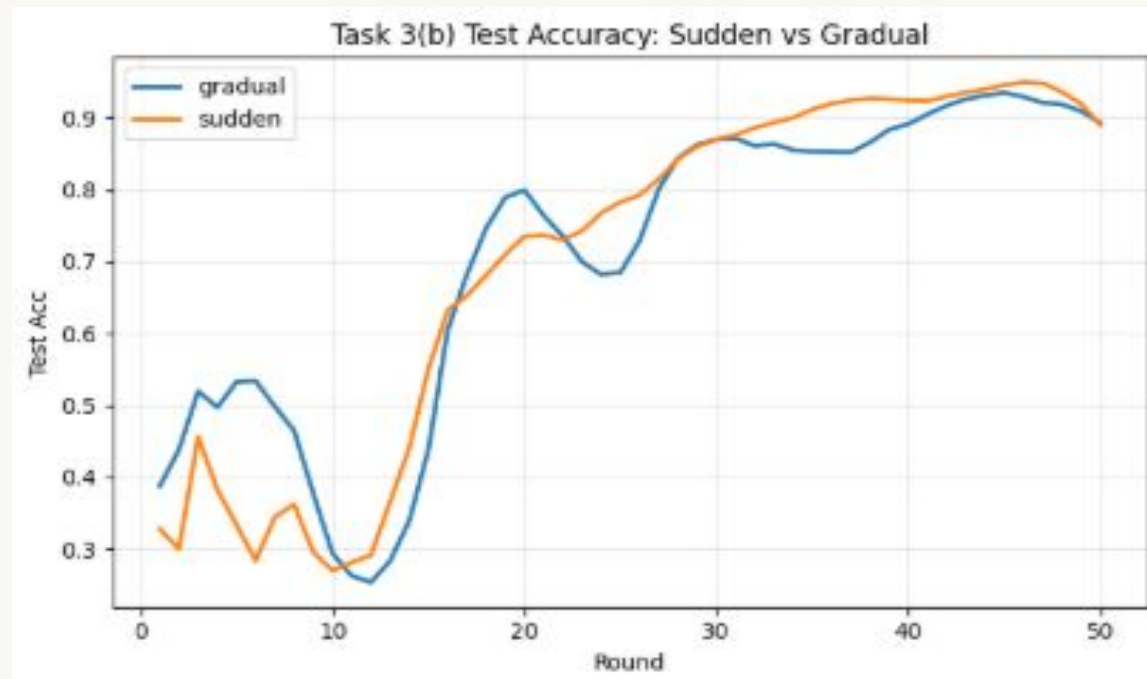
Task 2(Gradual)

- High fluctuations (slow data evolution)
- Performance varies across rounds
- Delayed adaptation to drift
- Instability from continuous distribution change



Task 3a

- Clustering reduces conflicting updates
- Similar clients grouped together
- FedAdam improves convergence
- More stable training



Task 3b

- The model maintains high accuracy (~90%+) for both sudden and gradual drift, showing strong generalization under non-stationary data
- The model quickly recovers after abrupt changes, indicating effective identification of relevant clients
- Performance improves steadily as the distribution shifts slowly over time
- Fluctuations are significantly lower due to drift-aware selection and clustering
- Grouping similar clients reduces conflicting updates and stabilizes training

Future Directions

Dynamic weighting of scoring factors : Instead of using fixed weights for multiple selection criteria, dynamically adjust the weights based on drift intensity, client reliability, or recent validation performance, allowing the model to prioritize more relevant factors over time

Dynamic client participation (K) : Adapt the number of selected clients per round based on the level of drift, selecting more clients during high drift to capture distribution changes and fewer during stable phases to reduce communication overhead

The End